Computer Science II Project Report

Oliwier Frączek 345307

June 6, 2026

1 Purpose of project

The goal of this project was to create a program capable of calculating various parameters of circuits powered by alternating current (AC). The program models electrical components and circuits using object-oriented programming principles, specifically inheritance and polymorphism. It computes impedance and power loss for individual components as well as composite circuits (series and parallel configurations).

2 Class design

The program is built around a class hierarchy that is derived from a base class `Component`. All electrical components inherit from this base class, which defines three pure virtual functions that every derived class must implement:

```
virtual void printType()

virtual std::complex<double> impedance(double frequency)

virtual double powerLoss(double current, double frequency)
```

Through polymorhpism, different component types can be stored in a single container and their impedance or power loss can be computed without knowing their type at compile time.

The hierarchy consists of the following classes:

2.1 Basic components

Three concrete classes representing ideal electrical elements:

`Resistor` Represents resistance R measured in ohms (Ω). Its impedance is purely real and independent of frequency.

`Capacitor` Represents capacitance C measured in farads (F). Its impedance is purely imaginary and frequency-dependent:

$$Z_C = -j \cdot \frac{1}{2\pi f C} \tag{1}$$

At DC ($f = 0$), the capacitor behaves as an open circuit. As frequency approaches infinity, impedance approaches zero (short circuit).

`Inductor` Represents inductance L measured in henries (H). Its impedance is purely imaginary and positive:

$$Z_L = j \cdot 2\pi f L \tag{2}$$

As f approaches infinity, so does the entire expression, which means the impedance grows and eventually becomes infinite (open circuit), and this is consistent with physical intuition.

2.2 Composite components

Two classes representing combinations of basic components:

`SeriesCircuit` A container that holds a collection of component pointers. The total impedance is the sum of all individual impedances:

$$Z_{\text{series}} = \sum_i Z_i \tag{3}$$

`ParallelCircuit` A container for components connected in parallel. The equivalent impedance is given by:

$$Z_{\text{parallel}} = \left(\sum_i \frac{1}{Z_i} \right)^{-1} \tag{4}$$

The full class hierarchy can be summarized as follows:

Class	**Parent**	**Description**
Component	-	Abstract base class
Resistor	Component	Real impedance $Z = R$
Capacitor	Component	Imaginary impedance Z_C
Inductor	Component	Imaginary impedance Z_L
SeriesCircuit	Component	$\sum Z_i$
ParallelCircuit	Component	$(\sum 1/Z_i)^{-1}$

These composite components essentially act as one big component with special logic.

3 Results of measurements

The program was tested with the following component values and circuit configuration:

Table 1: Input parameters

Parameter	Value	Unit
Frequency (f)	50.0	Hz
Current (I)	2.0	A
R_1	100.0	Ω
R_2	200.0	Ω
C_1	10.0	μF
L_1	50.0	mH

Results of the simulation are as follows:

Table 2: Computed circuit parameters

Circuit	Impedance Z	Power loss P (W)
SeriesCircuit(R1, C1)	(100 − 318.31) Ω	400W
ParallelCircuit(L1, R2)	(1.22614 15.6117) Ω	4.90455W

The series circuit combining the resistor and capacitor produces a negative imaginary component in impedance, consistent with capacitive reactance. The parallel circuit of the inductor and resistor yields a much smaller magnitude impedance due to the parallel configuration of the components.

4 C++ features used

4.1 Pointers and dynamic allocation

The program primarily uses raw pointers. Components are created on the stack but stored as pointers to base class:

```
std::vector<Component*> circuits = {&series, &parallel};
```

4.2 Inheritance and polymorphism

All concrete component types inherit from the abstract `Component` class, using virtual function overriding for behaviour definition:

```
virtual std::complex<double> impedance(double frequency) const override;
```

This allows for iterating over a collection of components of various types, and calling the same method without type checking.

4.3 Complex number support

The program uses `std::complex<double>` from $<$ `complex` $>$ to represent impedance in rectangular form (R, jX), where R is resistance and X is reactance. This avoids manual handling of real and imaginary parts separately.

4.4 Pure virtual functions

The base class `Component` contains no data members, it's a scaffold for building classes derived from it. All three member functions are pure virtual, making the class abstract and preventing instantiation directly:

```
virtual  Component() = default; // throws a compiler error
```

The destructor is also declared virtual to ensure proper cleanup when deleting through base pointers.

4.5 Operator overloading

A notable feature of this design is the use of operator overloading on the base class, demonstrated in main.cpp:

```
r1->impedance(frequency)
```

The arrow operator is implicitly overloaded via pointers to enable member access through Component*, allowing clean syntax when accessing component properties.

5 AI detection

This project will be put through an AI detection tool - I would like to talk about the efficacy of those tools. First of all, the vast majority of those tools are glorified data harvesting schemes. They use the collected data to train "AI humanizing" tools which attempt to make AI generated content appear more human, or simply sell the data to companies which will do the same. I wanted to perform an experiment, to that end, I have fed my project to an AI agent in order to create a detailed cleanroom rewrite spec. Using this spec, I then asked another agent to rewrite the whole project from scratch. The 2nd agent never saw the original source code - only the specification document. Any AI detection tools worth their salt should be able to easily distinguish between the two, if they cannot - that just goes to show they are completely useless at what they purport to do.